Day 10: Queues, Retries & Dead-Letter Handling

Safely processing asynchronous work when downstream services are slow or fail.

Why Queues Matter

- Decouple producers & workers
- Smooths traffic spikes
- Shields core systems

Retries & DLQ

- Bounded retries for failures
- Dead-letter queue for poison jobs
- Solves failure loops before they grow

Delivery Guarantees & Idempotency

- At-least-once delivery typical
- Duplicates require idempotent handlers
- Deduplication keys prevent reprocessing

Backpressure

- Controls queue growth
- Regulates producer speed
- Prevents consumer overload

Producer/Consumer Decoupling

- Producer submits jobs to queue
- Worker pulls & processes jobs at own pace
- Asynchronous, no blocking

At-Least-Once Delivery

- Deliver at least once per job
- Retries manage transient failures
- Handle duplicates with idempotent logic.

Producer/Consumer Decoupling

- Producer submits jobs to queue
- Worker pulls & processes jobs at own
- Asynchronous, no blocking

At-Least-Once Delivery

- Deliver at least once per job
- Retries manage transient failures
- Handle duplicates with idempotent logic.

Queues and retries make async work resilient to temporary failures.

Day 10: Queues, Retries, and Dead-Letter Handling

Day 10 focuses on how systems process asynchronous work safely when downstream services fail or become slow.

It covers:

- asynchronous job queues
- producer/consumer decoupling
- at-least-once delivery and duplicate handling
- retries with bounded attempts
- dead-letter queue (DLQ) for poison messages
- backpressure and queue capacity control

In short, Day 10 is about making background processing resilient without blocking user-facing request paths.

Core ideas

Why queues matter

- A `queue` lets producers hand off work and return quickly.
- Consumers can process jobs at their own speed.
- This decoupling smooths traffic spikes and protects core systems from bursts.

Delivery guarantees and idempotency

- Most practical queue systems provide `at-least-once` delivery.
- At-least-once means duplicates are possible, so handlers must be idempotent.
- Exactly-once is expensive and usually replaced by idempotent processing plus dedup keys.

Retries and DLQ

- Transient failures should be retried with bounded attempts.
- Permanent failures should move to a `dead-letter queue` for inspection.
- DLQ prevents one bad message from blocking healthy traffic.

Backpressure

- If producers are faster than consumers, queue length grows.
- Systems need queue limits, admission control, or load shedding.
- Backpressure keeps downstream services and workers stable under load.

Day-10 sample: Retry Queue with Dead-Letter Handling

This repository includes a small retry queue sample in both Python and Java.

Functional requirements

- Enqueue jobs with bounded queue capacity
- Poll jobs for processing
- Acknowledge successful jobs
- Retry failed jobs up to a configured maximum
- Move exhausted jobs to dead-letter storage

High-level components

- `Producer`: submits jobs into the queue
- `Worker`: polls jobs and processes them
- `In-flight map`: tracks currently delivered jobs awaiting ack/fail
- `Retry logic`: requeues failed jobs until max retries
- `Dead-letter list`: stores jobs that exceeded retry budget

Data flow

1. Producer adds jobs to the queue
2. Worker polls one job and starts processing
3. On success, worker acknowledges and job is removed
4. On failure, queue increments attempt count
5. If attempts stay within limit, job is requeued
6. If retry budget is exhausted, job is moved to dead-letter storage

Project structure

```
Day-10/
  README.md
  retry-queue-sample/
    python/
      retry_queue.py
      demo.py
    java/
      RetryQueue.java
      Main.java
```

Run the sample

Python

```
cd retry-queue-sample\python
python demo.py
```

Java

```
cd retry-queue-sample\java
javac Main.java RetryQueue.java
java Main
```

Interview takeaways

- Queues decouple request paths from slow background work
- At-least-once delivery requires idempotent consumers
- Retries need a cap to avoid infinite poison-message loops
- DLQs are critical for operability and debugging
- Backpressure controls are part of correctness, not just performance

Next improvements

- Add exponential backoff with jitter between retries
- Add delayed queues for scheduled retry attempts
- Add per-job deduplication keys
- Add metrics for queue depth, retries, and DLQ rate
- Replace in-memory storage with Redis, SQS, RabbitMQ, or Kafka

Source Code Appendix

Day-10/retry-queue-sample/java/Main.java

```
public class Main {
    public static void main(String[] args) {
        RetryQueue<String> queue = new RetryQueue<>(1, 2);

        System.out.println("enqueue invoice-101 -> " + queue.enqueue("invoice-101"));
        System.out.println("enqueue invoice-102 -> " + queue.enqueue("invoice-102"));
        System.out.println("enqueue invoice-103 (capacity exceeded) -> " + queue.enqueue("invoice-103"));
        printState(queue, "state after enqueue");

        RetryQueue.Job<String> first = queue.poll();
        if (first != null) {
            System.out.println("poll -> " + first.getJobId() + " " + first.getPayload());
            System.out.println("ack -> " + queue.ack(first.getJobId()));
        }
        printState(queue, "state after first job success");

        RetryQueue.Job<String> second = queue.poll();
        if (second != null) {
            System.out.println("poll -> " + second.getJobId() + " " + second.getPayload());
            System.out.println("fail -> " + queue.fail(second.getJobId()));
        }
        printState(queue, "state after first failure");

        RetryQueue.Job<String> retry = queue.poll();
        if (retry != null) {
            System.out.println("poll retry -> " + retry.getJobId() + " " + retry.getPayload());
            System.out.println("fail again -> " + queue.fail(retry.getJobId()));
        }
        printState(queue, "final state");
    }

    private static void printState(RetryQueue<String> queue, String label) {
        System.out.println(label);
        System.out.println("  queue: " + queue.queueIds());
        System.out.println("  inflight: " + queue.inflightIds());
        System.out.println("  dlq: " + queue.deadLetterIds());
    }
}
```

Day-10/retry-queue-sample/java/RetryQueue.java

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class RetryQueue<T> {
    public static final class Job<T> {
        private final String jobId;
        private final T payload;
        private int attempts;

        private Job(String jobId, T payload) {
            this.jobId = jobId;
            this.payload = payload;
            this.attempts = 0;
        }

        public String getJobId() {
            return jobId;
        }
        public T getPayload() {
```

```
        return payload;
    }

    private int incrementAttempts() {
        attempts += 1;
        return attempts;
    }
}

private final int maxRetries;
private final int capacity;
private int counter;
private final Deque<Job<T>> queue;
private final Map<String, Job<T>> inflight;
private final List<Job<T>> deadLetter;

public RetryQueue(int maxRetries, int capacity) {
    if (maxRetries < 0) {
        throw new IllegalArgumentException("maxRetries must be >= 0");
    }
    if (capacity <= 0) {
        throw new IllegalArgumentException("capacity must be positive");
    }
    this.maxRetries = maxRetries;
    this.capacity = capacity;
    this.counter = 0;
    this.queue = new ArrayDeque<>();
    this.inflight = new HashMap<>();
    this.deadLetter = new ArrayList<>();
}

public String enqueue(T payload) {
    if (queue.size() >= capacity) {
        return null;
    }

    counter += 1;
    Job<T> job = new Job<>("job-" + counter, payload);
    queue.addLast(job);
    return job.getJobId();
}

public Job<T> poll() {
    Job<T> job = queue.pollFirst();
    if (job == null) {
        return null;
    }
    inflight.put(job.getJobId(), job);
    return job;
}

public boolean ack(String jobId) {
    return inflight.remove(jobId) != null;
}

public String fail(String jobId) {
    Job<T> job = inflight.remove(jobId);
    if (job == null) {
        return "missing";
    }

    int attempts = job.incrementAttempts();
    if (attempts <= maxRetries) {
        queue.addLast(job);
        return "requeued";
    }

    deadLetter.add(job);
    return "dead-lettered";
}
public List<String> queueIds() {
```

```
        return queue.stream().map(Job::getJobId).collect(Collectors.toList());
    }

    public List<String> inflightIds() {
        return new ArrayList<>(inflight.keySet());
    }

    public List<String> deadLetterIds() {
        return deadLetter.stream().map(Job::getJobId).collect(Collectors.toList());
    }
}
```

Day-10/retry-queue-sample/python/demo.py

```
from retry_queue import RetryQueue


def print_state(queue: RetryQueue[str], label: str) -> None:
    print(label)
    print("  queue:", queue.queue_ids())
    print("  inflight:", queue.inflight_ids())
    print("  dlq:", queue.dead_letter_ids())


def main() -> None:
    queue = RetryQueue[str](max_retries=1, capacity=2)

    print("enqueue invoice-101 ->", queue.enqueue("invoice-101"))
    print("enqueue invoice-102 ->", queue.enqueue("invoice-102"))
    print("enqueue invoice-103 (capacity exceeded) ->", queue.enqueue("invoice-103"))
    print_state(queue, "state after enqueue")

    first = queue.poll()
    if first:
        print("poll ->", first.job_id, first.payload)
        print("ack ->", queue.ack(first.job_id))
    print_state(queue, "state after first job success")

    second = queue.poll()
    if second:
        print("poll ->", second.job_id, second.payload)
        print("fail ->", queue.fail(second.job_id))
    print_state(queue, "state after first failure")

    retry = queue.poll()
    if retry:
        print("poll retry ->", retry.job_id, retry.payload)
        print("fail again ->", queue.fail(retry.job_id))
    print_state(queue, "final state")


if __name__ == "__main__":
    main()
```

Day-10/retry-queue-sample/python/retry_queue.py

```
from __future__ import annotations

from collections import deque
from dataclasses import dataclass, field
from typing import Deque, Dict, Generic, TypeVar

T = TypeVar("T")


@dataclass
class Job(Generic[T]):
    job_id: str
    payload: T
```

```
    attempts: int = 0


@dataclass
class RetryQueue(Generic[T]):
    max_retries: int
    capacity: int
    _counter: int = field(default=0, init=False)
    _queue: Deque[Job[T]] = field(default_factory=deque, init=False)
    _inflight: Dict[str, Job[T]] = field(default_factory=dict, init=False)
    _dead_letter: list[Job[T]] = field(default_factory=list, init=False)

    def __post_init__(self) -> None:
        if self.max_retries < 0:
            raise ValueError("max_retries must be >= 0")
        if self.capacity <= 0:
            raise ValueError("capacity must be positive")

    def enqueue(self, payload: T) -> str | None:
        if len(self._queue) >= self.capacity:
            return None

        self._counter += 1
        job = Job(job_id=f"job-{self._counter}", payload=payload)
        self._queue.append(job)
        return job.job_id

    def poll(self) -> Job[T] | None:
        if not self._queue:
            return None

        job = self._queue.popleft()
        self._inflight[job.job_id] = job
        return job

    def ack(self, job_id: str) -> bool:
        return self._inflight.pop(job_id, None) is not None

    def fail(self, job_id: str) -> str:
        job = self._inflight.pop(job_id, None)
        if job is None:
            return "missing"

        job.attempts += 1
        if job.attempts <= self.max_retries:
            self._queue.append(job)
            return "requeued"

        self._dead_letter.append(job)
        return "dead-lettered"

    def queue_ids(self) -> list[str]:
        return [job.job_id for job in self._queue]

    def inflight_ids(self) -> list[str]:
        return list(self._inflight.keys())

    def dead_letter_ids(self) -> list[str]:
        return [job.job_id for job in self._dead_letter]
```